

WB Brands Finance OS

Problem & Remediation Report - what's broken, and how long it takes to fix together

A candid, verified inventory of the real problems in the codebase today - correctness, data integrity, security, and the accounting model - each explained in plain English, rated by severity, and sized for a collaborative fix where Claude writes the code and you act as the brain: making the finance decisions, applying migrations to the live database, and testing against real numbers.

Branch	classification__rules_category
Prepared	2026-07-10
Findings	22 verified problems across 4 themes
Method	Multi-agent code audit, adversarially verified

Every finding below was located in the actual code and cross-checked. File:line references are included so each can be verified independently. Effort estimates are collaborative working time, explained on the next page.

How to read this

There are 22 verified problems, grouped into four themes: the accounting model, data integrity, security/authorization, and schema/docs hygiene. Each has a plain-English explanation of what it is, why it matters to you, exactly where it lives in the code, and what fixing it involves.

Severity



- CRITICAL - exploitable or corrupting now; fix first.
- HIGH - wrong numbers or real security exposure in normal use.
- MEDIUM - correctness/security gap under specific conditions.
- LOW - hygiene; cheap to clear, low blast radius.

What 'effort' means here

Estimates are in collaborative working time - us at the keyboard together - not wall-clock calendar time. The important thing to understand: my code-writing is the SMALL part. The clock is really driven by three things that are yours, not mine:

- Decisions - above all, cash-basis vs accrual (Phase 3/4). One conversation with your CPA unblocks the whole accounting rebuild.
- Applying migrations to the live database - done carefully, with backups, because it's real financial data and hard to undo.
- Testing against real numbers - confirming a fixed report actually ties out, which only you can judge.

So a '0.5 day' item is often ~30 minutes of my coding plus your review; a '2-3 weeks' item is genuinely large because it touches money, migrations, and a backfill of historical data that we must verify. Ranges are honest, not padded - the low end assumes decisions are ready and the high end assumes we hit real-data surprises (we usually do).

Good news up front: the codebase typechecks cleanly (strict TypeScript, zero errors), the service key is not exposed to the browser, no secrets are committed, and page-level route gating works. The problems are real but bounded - none require a rewrite.

The problems

Theme 1 - Accounting model & correctness

1. The books are single-sided and cash-basis - not real double-entry

HIGH **EFFORT:** see Phase 3

What it is: Every classified bank line is stored as ONE row with a single account and a signed amount. The opposite ('cash') side of each transaction is never recorded. Real double-entry accounting records both sides of every event so that total debits always equal total credits. You already have the right tables (journal_entries + ledger_entries) and createJournal even balance-checks manual entries - but ordinary bank activity never flows through them.

Why it matters: Nothing guarantees the books balance. Cash on the balance sheet is tracked separately from the ledger, so the two can drift with no error. It also blocks accrual accounting, AP/AR tie-out, and a trustworthy trial balance - the foundation everything else in this report sits on.

`actions/transactions.ts:62 (signed one-leg post) | lib/queries/reports.ts (totals sum one-sided amounts)`

Fix: Route every posting through a balanced journal entry (debit + credit) via a Postgres function, add a bank-account -> chart-of-accounts mapping for the contra leg, link each transaction to its journal entry, and derive reports from the ledger. Detailed in Phase 3.

2. Accounts Payable and Accounts Receivable are disconnected from the ledger

HIGH **EFFORT:** see Phase 4

What it is: Vendors, bills (invoices) and ap_items exist as standalone trackers. Paying a bill just flips a 'paid' flag; recording a payment just bumps a number. None of it posts to the general ledger, and there is no customer/AR side at all - only a bare account code 1100.

Why it matters: AP and AR balances can't be trusted or reconciled against the GL, aging is cosmetic, and you can't tell what you truly owe or are owed. Once double-entry exists, a vendor payment must reduce AP (not re-book an expense) and a customer deposit must reduce AR (not re-book revenue) - otherwise expenses and revenue get double-counted.

`actions/ap.ts:13 (payApItem flips paid) | actions/invoices.ts:60 (recordPayment bumps amount_paid)`

Fix: Turn AP/AR into true subledgers that post to control accounts (2010 AP, 1100 AR) and add a 'this bank line settles invoice X' matching mode. Build a customers table + AR invoicing. Phase 4.

3. Manual edits can silently desync the two representations of a transaction

MEDIUM **EFFORT:** 2-3 days

What it is: createJournal mirrors each journal line into the single-sided transactions table (memo je:<id>). But editTransaction can change that mirrored amount without touching the underlying ledger_entries row, so the two copies quietly disagree.

Why it matters: A single edit can make the P&L (read from transactions) and the journal (read from ledger_entries) report different numbers for the same event, with no warning.

`actions/journals.ts:44 (mirrors into transactions) | actions/transactions.ts:636 (editTransaction)`

Fix: Once postings are unified on the ledger (Phase 3) this largely disappears; until then, block edits on

mirrored rows or update both sides atomically.

4. Consolidated statements don't eliminate intercompany accounts

MEDIUM**EFFORT: 2-4 days**

What it is: The chart of accounts flags intercompany lines (`is_elimination` on 1400/2300/4950/6700), but that flag was dropped from the app's account type and is never read. Consolidated views 'trust that intercompany balances net out' - there is no elimination logic anywhere.

Why it matters: A consolidated P&L that includes both intercompany commission income (4950) and expense (6700) overstates top-line revenue and total expense by the intercompany amount; the consolidated balance sheet double-counts the intercompany receivable/payable.

`legacy/seed.sql:28,32,43,62 (is_elimination) | lib/queries/reports.ts:75-79 (acknowledges the gap)`

Fix: Re-expose `is_elimination` on the account type and subtract flagged accounts (or post explicit elimination journals) when building any consolidated / group view.

5. Per-entity P&L silently drops journal-entry adjustments

MEDIUM**EFFORT: 0.5-1 day**

What it is: Reports filter `journal_entries` on the text 'entity' column, but seeded and legacy journals only populated 'entity_id' (the text column is NULL). So when you pick a single entity, every accrual, elimination and adjusting entry with a NULL text-entity is excluded.

Why it matters: Accruals and adjustments vanish from every entity-scoped P&L and appear only in the fully consolidated view - so single-entity numbers are quietly wrong.

`lib/queries/reports.ts:123-128,172-176 (filters on entity text) | legacy/seed.sql:119 (entity_id only)`

Fix: Backfill entity (text) from `entity_id` on existing journals and standardize new writes to set both, or filter journals by `entity_id` everywhere.

Theme 2 - Data integrity & corruption risk

6. Re-importing a bank file double-counts everything in it

HIGH**EFFORT: 1-2 days**

What it is: Imports insert raw rows with no dedup key. The live table has an `external_id` column meant to catch duplicates, but the code never sets it. Because the downstream dedup checksum folds in the raw row's id (to keep genuinely-recurring charges distinct), two imported copies get different ids and both survive.

Why it matters: Re-uploading the same month, or an overlapping date range, books every transaction twice and inflates revenue, expense, and every P&L / balance-sheet figure - with no error and nothing to catch it.

`actions/import.ts:135-153 (no external_id / no conflict target) | supabase 0017 (checksum folds raw id)`

Fix: Populate and dedup on (source, `external_id`) at import, and add a one-time cleanup for any duplicate rows already imported. Needs care against real data - your review/testing is the bottleneck.

7. 'Mark as CC Payment' is fully broken on the live database

HIGH**EFFORT: 0.5 day**

What it is: The new code requires suspense account 2999 to exist and throws if it doesn't. That account is created by migration 0019, which per the plan doc is not yet applied to the live DB.

Why it matters: Right now, on production, clicking 'Mark as CC Payment' errors out and does nothing - rows aren't classified, moved, or posted. The feature is dead until the migration is applied. (It fails safely - no corruption.)

```
actions/transactions.ts:441-451 (throws if 2999 missing) |  
plan/credit-card-settlements-separation.md:20
```

Fix: Apply migration 0019 to the live DB and verify the CC inbox end-to-end. Mostly your migration-apply + a quick verification pass.

8. Picking entities SP or RUSH shows every entity's data instead

MEDIUM**EFFORT: 0.5-1 day**

What it is: The code lists 8 entity codes but the database only has rows for 6 (SP and RUSH were never seeded). One of the two entity-filter paths, when it can't resolve a code to an id, returns the query UNFILTERED rather than empty.

Why it matters: Selecting 'SP Brands' or 'Rushmore' in the inbox shows all entities' unclassified transactions while claiming to show only that one - a data-leak and a correctness bug at once. Their rows also import with a NULL entity_id and can never be filtered or auto-tagged.

```
lib/entity-filter.ts:47-52 (returns unfiltered on empty) | lib/entities.ts:9-18 vs  
legacy/seed.sql:8-15
```

Fix: Seed the missing entities, and make the id-filter return zero rows (not all rows) for an unknown code. Small but high-value.

9. Two migrations share number 0018 and can't both be applied

HIGH**EFFORT: 0.5 day**

What it is: There are two files named 0018 (a text-typed and a date-typed checksum). Only the date-typed one is valid on production, but any standard runner applies BOTH in filename order and the first one aborts with 'generation expression is not immutable'. The disambiguation lives only in a code comment.

Why it matters: You cannot re-run migrations against production or any real environment - the migration history is ambiguous and self-contradictory about which checksum is actually live.

```
supabase/migrations/0018_..._stable.sql vs 0018_..._stable_date.sql
```

Fix: Delete/rename the invalid variant so exactly one 0018 exists, and note it in the migration log.

10. A fresh database cannot be rebuilt from the migrations

HIGH**EFFORT: 1-2 days**

What it is: The baseline migration (0001) is an empty placeholder. Most core tables (transactions, accounts, entities, journals, etc.) are never created by any migration - they exist only in the live DB. Applying the migrations to an empty database fails on the first policy/foreign-key reference.

Why it matters: There is no reproducible schema in source control. CI, staging, disaster recovery, and new-developer setup are all impossible from the repo; every environment is a hand-cloned copy of production.

supabase/migrations/0001_baseline_schema.sql:20-22 (intentionally empty)

Fix: Dump the live schema into 0001 (or a proper baseline migration) and prove a from-scratch rebuild in CI. Unblocks safe testing of everything else.

Theme 3 - Security & authorization

11. Any logged-in user can read or change ANY entity's data (entity-level IDOR)

CRITICAL**EFFORT: 5-10 days**

What it is: Authorization checks a user's ROLE but never which entities they're allowed to touch. There is no per-user entity scope at all. Nearly every action trusts a client-supplied entity code or row id, and the data client uses the service key (which bypasses the database's own RLS). This is the single largest issue and spans ~10 actions plus the AI route.

Why it matters: A bookkeeper or COO scoped in the UI to one entity can classify, post journals, pay bills, lock/unlock months, edit P&L, or pull AI summaries for a completely different entity just by sending a different code. For a multi-entity finance system this is the core security hole.

`actions/{transactions,journals,ap,pnl-manual,reconcile,classify,cashbook,period-close}.ts` | `app/api/ai/route.ts`

Fix: Add an entity-scope column to profiles + a `validateEntityAccess()` helper called after the role check, derive the true entity from the stored row (never trust the client), and build UI to assign each user's entities. The big security rock.

12. Anyone can promote themselves to admin

CRITICAL**EFFORT: 0.5 day**

What it is: The 'claim first admin' flow lets any authenticated user become admin whenever zero admins exist - it never checks the admin-email allowlist, and the check-then-write isn't atomic, so two users racing can both become admin.

Why it matters: During the bootstrap window (or any moment with no admin), an unintended or malicious account gains permanent full admin, defeating the allowlist entirely.

`app/no-role/actions.ts:41-79` (`claimFirstAdmin`)

Fix: Require the caller's email to be in the allowlist before promoting, and make it atomic (unique constraint or a single guarded write).

13. Role changes never take effect - you can't actually demote anyone

HIGH**EFFORT: 0.5-1 day**

What it is: The current-user lookup reads role from the profiles table first, but the admin Users page writes role only to auth `user_metadata`. `profiles.role` is set once at signup and never updated after.

Why it matters: Demoting a compromised or departing user from admin to a lower role silently does nothing - their elevated access persists. Promotions are equally ineffective. A real, current authorization bug.

`lib/auth/profile.ts:44-48` (`profiles first`) | `app/(app)/admin/users/actions.ts:129-155` (`writes metadata only`)

Fix: Write role changes to `profiles.role` (the authoritative source) as well as `/` instead of `metadata`, and log the change.

14. The database's own security trusts a field users can edit themselves

HIGH**EFFORT: 0.5-1 day**

What it is: The RLS role helper reads role from user_metadata first, which in Supabase the user can write to with the public anon key and their own session. (RLS isn't the live enforcement today, but this is a landmine for when it is turned on - and it's also an app-layer fallback.)

Why it matters: A user could set their own metadata role to 'admin' and, once RLS is relied upon, query admin-only tables directly against the API. It quietly undermines the intended DB backstop.

```
supabase/migrations/0002_enable_rls.sql:20-23      (coalesce      metadata      first)      |  
lib/auth/profile.ts:42-48
```

Fix: Move role to service-role-only app_metadata (or profiles) and reference that in both RLS and the app fallback.

15. The AI advisor leaks every entity's cash balances and invoices

HIGH**EFFORT: 0.5 day**

What it is: The AI context builder accepts an entity but fetches cash balances and open invoices for ALL entities with no filter, then feeds them to the model.

Why it matters: A user asking about one entity gets other entities' cash positions and invoices in the response - cross-entity data exposure through the assistant.

```
app/api/ai/route.ts:69-71 (listCashBalances / listOpenInvoices unfiltered)
```

Fix: Filter both queries by the requested entity (and, with #11, by the user's allowed entities).

16. A large file upload can crash the server (DoS)

MEDIUM**EFFORT: 0.25 day**

What it is: Import reads the entire uploaded file into memory and parses it, with the request-size limit set to 300MB and no file-size check in the action.

Why it matters: A logged-in user uploading a very large workbook can exhaust the serverless function's memory and take the import path down.

```
actions/import.ts:35,67 (buffers whole file) | next.config.ts (bodySizeLimit 300mb)
```

Fix: Reject files above a sane limit (e.g. 25-50MB) before parsing; ideally stream CSVs.

17. A read-only CPA can write P&L entries

MEDIUM**EFFORT: 0.25 day**

What it is: P&L manual-entry create/delete are gated on 'can view the P&L page' rather than on write roles, and the view list includes CPA - who is meant to be read-only on financials.

Why it matters: A CPA account can create and delete manual P&L figures, contrary to the intended read-only role.

```
actions/pnl-manual.ts:20,69 (canViewPage gate) | lib/auth/permissions.ts:60
```

Fix: Gate these on the write-role set via requireRole, matching every other mutating action.

18. No audit trail on P&L changes or user/role administration

MEDIUM**EFFORT: 0.5-1 day**

What it is: P&L manual entries write no audit log and aren't covered by the DB trigger; creating/deleting users and changing roles also log nothing. The DB-level trigger records no actor for app writes (the service client has no session).

Why it matters: The most sensitive actions - editing financials and granting/removing access - are unattributable, which undermines both security investigations and CPA/audit requirements.

`actions/pnl-manual.ts` (no `writeAuditLog`) | `app/(app)/admin/users/actions.ts` (no `writeAuditLog`)

Fix: Call `writeAuditLog` on these mutations and add `pnl_manual_entries` to the trigger's table list.

19. Open redirect on login

LOW**EFFORT: 0.25 day**

What it is: The login 'next' parameter is accepted if it starts with '/', but a protocol-relative value like '//evil.com' passes that check and redirects off-site.

Why it matters: Aids post-login phishing - a crafted login link can bounce an authenticated user to an attacker site.

`app/login/actions.ts:44-49`

Fix: Also reject values starting with '//' (or validate same-origin with the URL API).

Theme 4 - Schema & documentation hygiene

20. A database type claims a column that doesn't exist

MEDIUM**EFFORT: 0.25 day**

What it is: The classification-rule type declares `vendor_id`, but the live table has no such column (writing it is rejected at runtime). The vendor-tagging path is therefore dead, and the type invites a future write that will fail only at runtime, not at compile time.

Why it matters: Silent dead functionality now, and a latent runtime error trap for whoever touches rules next.

`lib/supabase/types.ts:131` (`vendor_id`) | `actions/classify.ts:33-35` (comment: `PGRST204 if sent`)

Fix: Remove `vendor_id` from the type (and input schema), or add the column if vendor-tagging is actually wanted.

21. The project docs contradict what the code actually does

LOW**EFFORT: 0.25 day**

What it is: `CLAUDE.md` says RLS is not enabled and the Anthropic key isn't wired - but RLS policies are deployed and the AI advisor is fully live. A client-layer comment claims RLS policies protect the data, which is misleading given the service key bypasses them.

Why it matters: Future readers (and audits) will misjudge the real security posture and wiring, leading to wrong decisions.

`CLAUDE.md` (RLS / Anthropic notes) | `lib/supabase/data.ts:14-17` (comment)

Fix: Reconcile the docs and comments with reality in one small pass.

22. The data client silently falls back to the anon key

LOW EFFORT: 0.25 day

What it is: If the service-role key is unset, the server data client quietly uses the public anon key instead of failing.

Why it matters: A misconfigured environment would run with weaker-than-intended DB access and no error to signal it.

`lib/supabase/data.ts:28` (?? anon key)

Fix: Fail loudly at startup if the service-role key is missing on the server.

Roadmap & total effort

The five phases below are ordered by urgency and dependency. Phases 0-2 (roughly 3-4 weeks of collaborative time) are independent of the accounting rebuild and could even run alongside it. Phases 3-4 are the large accounting work and depend on the cash-vs-accrual decision - and I'd want Phase 2's reproducible schema in place first, so we can test the rebuild safely.

Phase 0 - Stop the bleeding

3-4 days

The things that are actively wrong right now: apply migration 0019 (#7), fix re-import double-counting (#6), resolve the duplicate 0018 migration (#9), close the self-promote-to-admin hole (#12), and stop the AI cross-entity leak (#15). Small, high-urgency, mostly independent.

Covers findings: #7, #6, #9, #12, #15

Phase 1 - Security hardening

8-13 days

The entity-level IDOR class (#11) is the big rock here and drives most of the time; alongside it, fix the broken role changes (#13), the user-editable RLS role field (#14), the read-only-CPA write gate (#17), the missing audit trail (#18), the DoS upload (#16), and the open redirect (#19).

Covers findings: #11, #13, #14, #16, #17, #18, #19

Phase 2 - Data integrity & repo health

4-6 days

Make the schema reproducible (#10), fix the SP/RUSH filter leak (#8), recover the dropped per-entity journal adjustments (#5), and clear the schema/doc hygiene items (#20, #21, #22). This unblocks safe testing of everything else.

Covers findings: #10, #8, #5, #20, #21, #22

Phase 3 - Double-entry accounting engine

2-3 weeks

The core rebuild: bank-account -> chart-of-accounts mapping, an atomic balanced-posting function, a balance-enforcing constraint, link each transaction to its journal entry, rewrite reports off the ledger, and backfill the historical contra legs. Fixes #1 and #3 and makes the books provably balance. Gated by one decision from you + your CPA: cash-basis vs accrual.

Covers findings: #1, #3

Phase 4 - Connect AP / AR + intercompany

3-4 weeks

Turn AP and AR into real subledgers tied to control accounts, add invoice/bill settlement matching against bank lines, build the missing customer + AR-invoicing side from scratch, and add intercompany elimination to consolidated views. Fixes #2 and #4.

Covers findings: #2, #4

Grand total: roughly 9-13 weeks of collaborative working time

Done thoroughly and sequentially. About 60% of that is the accounting engine + AP/AR (Phases 3-4). If the goal is just to make the system safe and trustworthy without the full accrual rebuild, Phases 0-2 alone (~3-4 weeks) get you a secure, non-corrupting, reproducible system - the accounting model rebuild can then be scheduled on its own once the CPA decision is made.

Suggested first move: knock out Phase 0 in our next session or two (it's fast and stops active harm), and in parallel book the cash-vs-accrual conversation so Phases 3-4 aren't blocked when we get there.